

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 1: FUNDAMENTOS BÁSICOS DE PYTHON (NIVEL 1
(PRINCIPIANTES))

Clase 07: Funciones Reutilizables y Modulares

«El Electrodoméstico y la Entrega del Cajero»

Modularización y abstracción: definición con def, parámetros posicionales y nombrados, valores por defecto, valor de retorno (return), ámbito (scope) y type hints.

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Principiante Absoluto | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: La Modularización y el Principio DRY	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Caja Negra Funcional y Ámbito de Variables (Scope)	Pág. 5
Capítulo 3	Implementación Práctica en Python: Módulo de Facturación con Funciones Tipadas	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Gotchas Clásicos con Funciones	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender el principio DRY (Don't Repeat Yourself), la diferencia entre print y return, y el scope local de variables.



Competencia Práctica

Escribir funciones modulares, documentadas con docstrings y fuertemente tipadas listas para producción.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. La Modularización y el Principio DRY

El código profesional no se escribe dos veces; cuando una lógica se necesita en múltiples lugares, se encapsula en una función.

☀ **Metáfora Central: El Electrodoméstico y la Entrega del Cajero**

Una función es como un electrodoméstico: tiene una ranura de entrada (parámetros), un motor interno que realiza una tarea específica, y una bandeja de salida donde entrega el resultado terminado (return).

Principios Teóricos y Modelo Mental

Parámetros vs Argumentos: Los parámetros son los nombres en la firma (def), los argumentos son los valores reales que pasas al invocarla.

Diferencia crucial: print() solo muestra texto en la pantalla pero devuelve None; return devuelve el valor a la variable que llamó a la función para seguir trabajando con él.

⚡ **Regla de Oro en Python**

Una función debe hacer una sola cosa y hacerla excepcionalmente bien (Principio de Responsabilidad Única).

2. Caja Negra Funcional y Ámbito de Variables (Scope)

Flujo de invocación, paso de argumentos, aislamiento de variables locales y retorno de valor.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	La función es llamada y se asignan los argumentos a los parámetros.	Creación del Call Stack Frame
2. Evaluación	Se ejecutan las instrucciones en un ámbito local aislado.	Variables locales temporales
3. Transformación	La instrucción 'return' finaliza la ejecución de la función y emite el resultado.	Envío del valor de retorno
4. Retorno / Salida	El stack frame se destruye y la memoria local se libera.	Retorno al flujo principal



Visualización Mental

Las variables creadas dentro de una función mueren cuando la función termina: nunca intentes acceder a una variable local desde fuera.

3. Módulo de Facturación con Funciones Tipadas

Diseño de funciones modulares con valores por defecto, docstrings y anotaciones de tipo:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
def calcular_total_factura(
    subtotal: float,
    tasa_impuesto: float = 0.21,
    descuento: float = 0.0
) -> dict[str, float]:
    """Calcula el desglose final de una factura comercial."""
    monto_descuento = subtotal * descuento
    base_imponible = subtotal - monto_descuento
    impuestos = base_imponible * tasa_impuesto
    total_pagar = base_imponible + impuestos

    return {
        "subtotal": subtotal,
        "descuento_aplicado": monto_descuento,
        "impuestos": impuestos,
        "total": round(total_pagar, 2)
    }

# Uso con argumentos por nombre (keyword arguments)
factura = calcular_total_factura(subtotal=150.0, descuento=0.10)
print(f"Total a pagar: ${factura['total']}")
```

Análisis del Código Fuente

Función pura con parámetros opcionales con valores predeterminados, tipado formal y retorno estructurado en diccionario.

4. Gotchas Clásicos con Funciones

Errores comunes de diseño y sintaxis en funciones de Python:

⚠ Gotcha Frecuente (Trampa de Principiante)

Usar argumentos mutables por defecto (como `def func(lista=[])`); la lista se comparte entre llamadas sucesivas.

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

```
def agregar(item, lista=[]): # ¡Peligro mutable!  
    lista.append(item)  
    return lista
```

✓ Patrón Pythonic / Correcto

```
def agregar(item, lista=None):  
    if lista is None: lista = []  
    lista.append(item)  
    return lista
```

🛡 Consejo de Resiliencia en Producción

Usa siempre `None` como valor predeterminado para parámetros que contengan estructuras mutables.

5. Resumen Ejecutivo y Conclusiones

Dominas el pilar de la abstracción y la reutilización de código mediante funciones profesionales.



Logro Alcanzado en esta Clase

Capacidad para escribir código limpio, modular, desacoplado y fácil de probar.

Notas del Instructor y Siguientes Pasos

En la Clase 08 integraremos los 7 temas en un Proyecto Integrador completo: el Gestor de Tareas en Consola.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Escribe una función recursiva que calcule el factorial de un número entero positivo con su caso base bien definido.